

CSE422 Lab Project Report

Retail Store Sales Discount Prediction and Customer Segmentation Analysis

Table of Contents

1. Introduction
 2. Dataset Description
 3. Dataset Pre-processing
 4. Dataset Splitting
 5. Model Training & Testing (Supervised & Unsupervised)
 6. Model Selection/Comparison Analysis
 7. Conclusion
-

1. Introduction

Project Overview

This project aims to analyze retail store sales data to predict whether customers will receive discounts on their purchases and to identify meaningful customer segments through unsupervised learning. The primary objectives are to build classification models that can accurately predict discount application based on transaction features and to discover natural groupings in customer purchasing behavior.

Problem Statement

Retail businesses face the challenge of optimizing their discount strategies to maximize both customer satisfaction and profitability. Understanding which transactions are likely to receive discounts and identifying distinct customer segments can help businesses make data-driven decisions about pricing strategies, inventory management, and targeted marketing campaigns.

Motivation

The motivation behind this project stems from the need for:

- **Predictive Analytics:** Enabling retailers to forecast discount patterns and optimize promotional strategies
- **Customer Segmentation:** Identifying distinct customer groups to tailor marketing efforts and improve customer experience
- **Business Intelligence:** Providing actionable insights into purchasing behaviors, payment preferences, and spending patterns
- **Revenue Optimization:** Balancing discount offerings with profitability through intelligent prediction models

This analysis will help retail businesses understand their customer base better and make informed decisions about discount allocation strategies while maintaining healthy profit margins.

2. Dataset Description

2.1 Basic Dataset Information

The retail store sales dataset contains comprehensive transaction information from a retail business. The dataset consists of:

- **Total Data Points:** 12,575 transactions
- **Total Features:** 11 columns (including the target variable)
- **Problem Type:** This is primarily a **Classification Problem** because our main target variable "Discount Applied" is categorical (binary: Yes/No or 1/0). Additionally, we can treat "Total Spent" as a regression target for secondary analysis.

Why Classification? The target variable "Discount Applied" has discrete binary values (True/False), making this a binary classification problem. We aim to predict whether a customer will receive a discount based on transaction features.

2.2 Feature Analysis

The dataset contains the following features:

Input Features:

1. **Transaction ID** (object) - Unique transaction identifier
2. **Customer ID** (object) - Unique customer identifier
3. **Category** (object) - Product category (Categorical)
4. **Item** (object) - Specific item purchased (Categorical)
5. **Price Per Unit** (float64) - Unit price of the item (Quantitative)
6. **Quantity** (float64) - Number of units purchased (Quantitative)
7. **Payment Method** (object) - Method of payment (Categorical)
8. **Location** (object) - Purchase location (Categorical)
9. **Transaction Date** (object) - Date of transaction (Temporal)

Output Features: 10. **Total Spent** (float64) - Total transaction amount (Quantitative - Regression target)
11. **Discount Applied** (object) - Whether discount was applied (Categorical - Classification target)

2.3 Feature Types

Quantitative Features (Numerical):

- Price Per Unit
- Quantity
- Total Spent

Categorical Features:

- Category
- Item
- Payment Method

- Location
- Discount Applied

Identifier Features (to be dropped):

- Transaction ID
- Customer ID
- Transaction Date

2.4 Categorical Variable Encoding

Do we need to encode categorical variables? YES

Reason: Machine learning algorithms require numerical input. Our dataset contains several categorical features (Category, Payment Method, Location) that must be converted to numerical format. We used **Label Encoding** for these features, which assigns a unique integer to each category. This approach is suitable for our models as we applied proper scaling afterward to prevent any ordinal bias.

2.5 Correlation Analysis

The correlation heatmap reveals important relationships between features:

Key Findings from Correlation Matrix:

Strong Positive Correlations:

- **Item & Price Per Unit (0.85):** Higher item numbers correlate with higher prices
- **Quantity & Total Spent (0.71):** More items purchased naturally leads to higher total spending
- **Price Per Unit & Total Spent (0.60):** Higher unit prices contribute to larger transactions
- **Item & Total Spent (0.55):** Certain items are associated with higher spending

Weak Correlations with Target Variable (Discount Applied):

- All features show very weak correlation with "Discount Applied" (< 0.01)
- This suggests that discount application may depend on complex, non-linear relationships or external business rules rather than simple linear relationships

Interpretation: The weak correlations between input features and the target variable indicate that:

1. Discount decisions are not based on simple linear relationships
2. Advanced machine learning models may be needed to capture complex patterns
3. The problem may be inherently difficult, explaining the moderate model performance observed later

2.6 Imbalanced Dataset Analysis

Class Distribution for "Discount Applied":

- **No Discount (0):** 8,356 instances (66.4%)
- **Discount (1):** 4,219 instances (33.6%)

Class Ratio: 1.98:1 (approximately 2:1)

Analysis: The dataset shows moderate class imbalance with about twice as many non-discount transactions as discount transactions. This imbalance can lead to:

- Models biased toward predicting "No Discount"
- Poor recall for the minority class (Discount)
- Misleading accuracy metrics

Solution Applied: We used **SMOTE (Synthetic Minority Over-sampling Technique)** to balance the training data, creating synthetic samples of the minority class to achieve a 1:1 ratio (6,685:6,685) in the training set.

2.7 Exploratory Data Analysis (EDA)

Target Variable Distribution: **Figure 4.1** shows the clear imbalance in discount application, with non-discounted transactions (8,356) being approximately twice as common as discounted ones (4,219). The regression target (Total Spent) shows a right-skewed distribution with most transactions in the \$75-100 range, indicating typical retail behavior where moderate purchases dominate with occasional high-value outliers.

Feature Relationships (Pairplot Analysis): **Figure 4.2** reveals the fundamental challenge of this prediction problem:

- No clear linear separation between discounted and non-discounted transactions in any feature combination
- Complete overlap of blue (no discount) and green (discount) points throughout the feature space
- Similar distributions across both classes for all numerical features
- Price Per Unit and Total Spent show expected strong positive correlation (0.60)
- Quantity and Total Spent show strong correlation (0.71) as expected mathematically

Key Insights:

1. **Discount application appears to be independent of observable transaction features** - the complete mixing of classes in the pairplot explains why all models achieved $AUC < 0.5$
2. Customer purchasing patterns are diverse across both discount and non-discount groups, supporting the need for clustering analysis
3. The moderate feature correlations (price-spending, quantity-spending) reflect expected relationships but provide no discriminative power for discount prediction
4. **Visual evidence confirms:** Discount decisions are likely driven by external factors (customer loyalty, campaigns, business rules) not captured in the available features, making accurate prediction impossible with current data

3. Dataset Pre-processing

3.1 Identified Faults

Problem 1: Missing Values

Issue: Multiple columns contained missing values:

- **Item:** 1,213 missing values (9.65%)

- **Discount Applied:** 4,199 missing values (33.39%)
- **Price Per Unit:** 609 missing values (4.84%)
- **Quantity:** 604 missing values (4.80%)
- **Total Spent:** 604 missing values (4.80%)

Impact: Missing values can lead to:

- Reduced dataset size if rows are dropped
- Biased model training
- Inability to make predictions for incomplete data
- Potential loss of valuable information

Solution Applied:

1. **Numerical Features** (Price Per Unit, Quantity, Total Spent): Filled missing values with the **median** of each column. Median is preferred over mean as it's robust to outliers.
2. **Categorical Feature (Item):**
 - Cleaned the item names by removing category suffixes (_PAT, _MILK, _BUT, etc.)
 - Extracted numerical item IDs
 - Filled missing values with 0
 - Converted to integer type
3. **Target Variable (Discount Applied):** Filled missing values with 0 (assuming no discount when not specified)

Justification: This approach preserves all 12,575 records while maintaining data integrity through sensible imputation strategies.

Problem 2: Irrelevant Features

Issue: The dataset contained identifier and temporal columns that don't contribute to prediction:

- Transaction ID (unique identifier)
- Customer ID (unique identifier)
- Transaction Date (temporal data requiring special handling)

Impact: Including these features would:

- Add noise to the models
- Increase computational complexity
- Risk data leakage through unique identifiers
- Complicate model interpretation

Solution Applied: Dropped these three columns from the dataset.

Justification: These features provide no predictive value for discount application and removing them simplifies the model while improving generalization.

Problem 3: Categorical Variables

Issue: Machine learning algorithms require numerical input, but we have categorical features:

- Category (8 unique product categories)
- Payment Method (3 unique methods)
- Location (2 unique locations)

Impact: Models cannot process text/categorical data directly.

Solution Applied: **Label Encoding** was applied to all categorical features:

- Each unique category was assigned a unique integer
- Category: 0-7 for different product types
- Payment Method: 0-2 for different payment types
- Location: 0-1 for Online/Store

Justification: Label encoding is efficient and appropriate when combined with feature scaling, which we applied in the next step.

Problem 4: Feature Scaling (Different Scales)

Issue: Features have different scales:

- Quantity: typically 1-10
- Price Per Unit: 10-50
- Total Spent: 10-500

Impact:

- Models like KNN and Neural Networks are sensitive to feature scales
- Features with larger magnitudes can dominate the model
- Slower convergence in gradient-based algorithms

Solution Applied: **StandardScaler** was used to standardize all features:

- Transformed features to have mean = 0 and standard deviation = 1
- Applied separately to training and test sets to prevent data leakage
- Formula: $z = (x - \mu) / \sigma$

Justification: Standardization ensures all features contribute equally to model training and improves convergence for gradient-based algorithms.

Problem 5: Class Imbalance

Issue: The target variable "Discount Applied" is imbalanced:

- Before SMOTE: [6,685 No Discount, 3,375 Discount]

- Ratio: 1.98:1

Impact:

- Models biased toward majority class
- Poor performance on minority class
- High accuracy but low recall for discount prediction

Solution Applied: SMOTE (Synthetic Minority Over-sampling Technique)

- Applied only to training data
- Generated synthetic samples for minority class
- After SMOTE: [6,685 No Discount, 6,685 Discount]
- Perfect 1:1 balance achieved

Justification: SMOTE creates synthetic samples intelligently by interpolating between existing minority samples, avoiding simple duplication and helping models learn minority class patterns better.

4. Dataset Splitting

4.1 Splitting Strategy

Split Type: **Stratified Train-Test Split** (for classification) and **Random Split** (for regression)

Train-Test Ratio:

- **Training Set:** 80% (10,060 samples)
- **Test Set:** 20% (2,515 samples)

4.2 Why Stratified Split?

For the classification problem (Discount Applied), we used **stratified splitting** to maintain the same class distribution in both training and test sets. This ensures:

- Representative samples in both sets
- Unbiased evaluation metrics
- Prevention of scenarios where one class is over/under-represented in test data

4.3 Split Results

Classification Split:

- `X_train_class`: (10,060, 6 features)
- `X_test_class`: (2,515, 6 features)
- `y_train_class`: (10,060,) - maintains 1.98:1 ratio
- `y_test_class`: (2,515,) - maintains 1.98:1 ratio

Regression Split:

- `X_train_reg`: (10,060, 6 features)

- X_test_reg: (2,515, 6 features)
- y_train_reg: (10,060,) - Total Spent values
- y_test_reg: (2,515,) - Total Spent values

Random State: Set to 42 for reproducibility

4.4 Validation Strategy

While we used a single train-test split, the training process employed:

- **Early stopping** in Neural Network to prevent overfitting
- **Class balancing** through SMOTE after splitting (crucial: applied only to training data to prevent data leakage)

Note: All preprocessing (scaling, SMOTE) was fit on training data only and then applied to test data to ensure no information leakage from test to train set.

5. Model Training & Testing (Supervised & Unsupervised)

5.1 K-Nearest Neighbors (KNN) - Classification

Model Configuration:

- n_neighbors = 5
- Algorithm: Auto-selected
- Trained on balanced data (after SMOTE)

Performance Metrics:

- **Training Accuracy:** 76.52%
- **Test Accuracy:** 51.61%
- **Precision:** 0.3303 (33.03%)
- **Recall:** 0.4301 (43.01%)
- **AUC Score:** 0.4923

Analysis: KNN shows significant overfitting with a 24.9% gap between training and test accuracy. The model memorized training patterns but failed to generalize. The AUC score of 0.49 indicates performance worse than random guessing (0.50), suggesting the model struggles to distinguish between classes in the feature space.

5.2 Logistic Regression - Classification

Model Configuration:

- max_iter = 1000
- class_weight = 'balanced'
- Solver: Default (lbfgs)
- random_state = 42

Performance Metrics:

- **Training Accuracy:** 50.87%
- **Test Accuracy:** 50.02%
- **Precision:** 0.3331 (33.31%)
- **Recall:** 0.4882 (48.82%)
- **AUC Score:** 0.4851

Analysis: Logistic Regression shows minimal overfitting (only 0.8% gap), indicating good generalization but poor overall performance. The model essentially performs at chance level (50% accuracy), suggesting the relationship between features and discount application is highly non-linear or dependent on unavailable features. The highest recall among models (48.82%) means it's better at identifying actual discount cases.

5.3 Neural Network (MLP) - Classification

Model Configuration:

- Architecture: Input → 100 neurons → 50 neurons → Output
- hidden_layer_sizes = (100, 50)
- max_iter = 500
- early_stopping = True
- validation_fraction = 0.1
- random_state = 42

Performance Metrics:

- **Training Accuracy:** 58.23%
- **Test Accuracy:** 48.59%
- **Precision:** 0.3167 (31.67%)
- **Recall:** 0.4597 (45.97%)
- **AUC Score:** 0.4725

Analysis: The Neural Network shows moderate overfitting (9.6% gap) and the worst test performance among all models. Despite its capacity for learning complex patterns, it failed to find meaningful relationships in the data. This suggests the features provided may not contain sufficient signal to predict discount application effectively, regardless of model complexity.

5.4 Decision Tree - Classification

Model Configuration:

- max_depth = 10
- min_samples_split = 10
- class_weight = 'balanced'

- random_state = 42

Performance Metrics:

- **Training Accuracy:** 59.51%
- **Test Accuracy:** 63.62% ★ **BEST**
- **Precision:** 0.3394 (33.94%)
- **Recall:** 0.0889 (8.89%) ✖ **WORST**
- **AUC Score:** 0.4943

Analysis: Decision Tree achieved the highest test accuracy (63.62%) and interestingly shows negative overfitting (-4.1% gap), meaning it performs better on test data than training data. However, it has the lowest recall (8.89%), indicating it misses 91% of actual discount cases. The model appears to be overly conservative, predicting "No Discount" for most cases, which inflates accuracy due to class imbalance in the test set.

5.5 K-Means Clustering - Unsupervised Learning

Objective: Treat the problem as unsupervised learning to discover natural customer segments.

Optimal Cluster Selection:

- Method: Elbow Method + Silhouette Score Analysis
- Tested k values: 2 to 10
- **Optimal k:** 2 clusters (maximum silhouette score)

Model Configuration:

- n_clusters = 2
- init = 'k-means++'
- random_state = 42
- n_init = 10

Clustering Results:

Cluster Distribution:

- Cluster 0: 5,030 samples (50.0%)
- Cluster 1: 5,030 samples (50.0%)
- Perfect 50-50 split

Cluster Characteristics (Standardized Mean Values):

Cluster 0 (Higher Value Customers):

- Category: -0.084 (diverse categories)
- Item: +0.944 (higher-numbered items)
- Price Per Unit: +0.883 (higher prices)
- Quantity: +0.002 (average quantity)

- Payment Method: -0.023 (varied payment methods)
- Location: +0.015 (slight online preference)

Cluster 1 (Lower Value Customers):

- Category: +0.069 (diverse categories)
- Item: -0.780 (lower-numbered items)
- Price Per Unit: -0.730 (lower prices)
- Quantity: -0.001 (average quantity)
- Payment Method: +0.019 (varied payment methods)
- Location: -0.012 (slight store preference)

PCA Visualization:

- Principal Component 1 & 2 explain 47.91% of variance
- Clusters show some separation but with considerable overlap
- Suggests moderate clustering structure in the data

Interpretation: The K-Means algorithm identified two natural customer segments:

1. **Premium Segment:** Customers purchasing higher-priced items
2. **Value Segment:** Customers purchasing lower-priced items

The clustering is primarily driven by price and item features, suggesting pricing tier is the main differentiator in customer behavior.

6. Model Selection/Comparison Analysis

6.1 Overall Performance Comparison

Model Rankings by Test Accuracy:

1. **Decision Tree:** 63.62% ★
2. **KNN:** 51.61%
3. **Logistic Regression:** 50.02%
4. **Neural Network:** 48.59%

6.2 Precision and Recall Analysis

Precision Comparison:

- Logistic Regression: 33.31% (Best)
- Decision Tree: 33.94%
- KNN: 33.03%
- Neural Network: 31.67% (Worst)

All models have similar, poor precision (~33%), meaning about 2 out of 3 "Discount" predictions are false positives.

Recall Comparison:

- Logistic Regression: 48.82% ★ (Best)
- Neural Network: 45.97%
- KNN: 43.01%
- Decision Tree: 8.89% ✗ (Worst)

Key Finding: There's a clear **Precision-Recall Tradeoff**:

- Decision Tree: High accuracy (63.62%) but terrible recall (8.89%)
- Logistic Regression: Lower accuracy (50.02%) but best recall (48.82%)

6.3 Confusion Matrix Analysis

Decision Tree (Best Accuracy):

- True Negatives: High (correctly identifies non-discounts)
- False Positives: Very low (rarely predicts discount incorrectly)
- False Negatives: Very high (misses most actual discounts)
- True Positives: Very low (identifies few actual discounts)
- **Behavior:** Extremely conservative, predicts "No Discount" most of the time

Logistic Regression (Best Recall):

- True Negatives: Moderate
- False Positives: Higher (more liberal with discount predictions)
- False Negatives: Moderate (balanced)
- True Positives: Higher (catches more actual discounts)
- **Behavior:** More balanced approach to both classes

KNN and Neural Network:

- Show similar patterns to Logistic Regression
- Moderate performance across all confusion matrix cells
- No clear advantage in any specific area

6.4 ROC Curve and AUC Score Analysis

AUC Scores:

- Decision Tree: 0.4943
- KNN: 0.4923
- Logistic Regression: 0.4851
- Neural Network: 0.4725

Critical Finding: All AUC scores are **below 0.5** (random classifier threshold), indicating:

- Models perform worse than random guessing at discriminating between classes
- The features provided have minimal predictive power
- Discount application may be driven by external factors not in the dataset

ROC Curve Interpretation: All ROC curves hover near the diagonal line (random classifier), with some sections even falling below it. This confirms that the relationship between available features and discount application is extremely weak or non-existent.

6.5 Overfitting Analysis

Overfitting Gap (Train Accuracy - Test Accuracy):

- **KNN:** +24.9% 🚩 **Severe Overfitting**
- **Neural Network:** +9.6% 🚩 **Moderate Overfitting**
- **Logistic Regression:** +0.8% ✅ **Excellent Generalization**
- **Decision Tree:** -4.1% ✅ **No Overfitting** (performs better on test!)

Analysis:

- KNN severely overfits due to memorizing training patterns
- Neural Network shows moderate overfitting despite early stopping
- Logistic Regression generalizes best but has poor overall performance
- Decision Tree's negative gap suggests it learned general patterns that work better on test data

6.6 Best Model Selection

No Single "Best" Model - depends on business objective:

For Highest Accuracy (General Performance):

- **Decision Tree** (63.62% accuracy)
- Use when: You need to minimize false positives and can afford to miss some discount opportunities

For Identifying Discount Cases (Recall Priority):

- **Logistic Regression** (48.82% recall)
- Use when: It's important to catch actual discount cases, even with more false alarms

For Balanced Performance:

- **KNN or Neural Network** (moderate on all metrics)
- Use when: You need balanced performance across precision and recall

6.7 Key Insights from Comparison

1. **Problem Difficulty:** All models struggle significantly, with best accuracy only 63.62%
2. **Feature Inadequacy:** AUC scores < 0.5 indicate the provided features have minimal predictive power
3. **No Clear Winner:** Each model has trade-offs between accuracy, precision, and recall

- 4. **Generalization:** Simpler models (Logistic Regression, Decision Tree) generalize better than complex ones (Neural Network)
 - 5. **Business Decision Required:** Model selection depends on whether false positives or false negatives are more costly to the business
-

7. Conclusion

7.1 Understanding the Results

Primary Findings:

- 1. **Poor Overall Model Performance:** The best test accuracy achieved was only 63.62% (Decision Tree), which is only marginally better than a naive baseline (66.4% by always predicting "No Discount"). This indicates that predicting discount application from the available transaction features is extremely challenging.
- 2. **Weak Feature-Target Relationships:** The correlation analysis showed that all features have correlations < 0.01 with the target variable, and AUC scores below 0.5 confirm that the provided features contain minimal predictive signal for discount application.
- 3. **Customer Segmentation Success:** The unsupervised K-Means clustering successfully identified two natural customer segments based primarily on pricing tiers (premium vs. value customers), explaining 47.91% of variance with 2 principal components.
- 4. **Model Trade-offs:** Different models excel at different metrics:
 - Decision Tree: Best accuracy but worst recall
 - Logistic Regression: Best recall and generalization but chance-level accuracy
 - KNN: Severe overfitting issues
 - Neural Network: Unable to learn meaningful patterns despite complexity

7.2 Performance Comments

What Worked:

- Data preprocessing pipeline successfully handled missing values, categorical encoding, and class imbalance
- SMOTE effectively balanced training data from 1.98:1 to 1:1 ratio
- Feature scaling and standardization were properly implemented
- Stratified splitting maintained class distribution

What Didn't Work:

- No model could learn meaningful patterns to predict discounts
- Complex models (Neural Network) didn't outperform simple ones (Logistic Regression)
- All precision values remained around 33%, indicating high false positive rates
- AUC scores consistently below random classifier threshold

Model-Specific Comments:

- **Decision Tree:** Achieved highest accuracy by being extremely conservative (predicting "No Discount" most of the time), but this strategy misses 91% of actual discount cases
- **Logistic Regression:** Most reliable with excellent generalization, but essentially performs at chance level
- **KNN:** Memorized training patterns but couldn't generalize at all
- **Neural Network:** Failed to find complex patterns despite having the capacity to do so

7.3 Why These Results?

Likely Reasons for Poor Performance:

1. **Missing Critical Features:** Discount decisions in retail are likely based on factors not captured in this dataset:
 - Customer loyalty status/membership level
 - Promotional campaign timing
 - Inventory clearance needs
 - Seasonal factors
 - Customer purchase history
 - Competitor pricing
 - Marketing channel source
2. **Business Rule-Based Decisions:** Discounts may be applied based on deterministic business rules (e.g., "Apply 10% off on orders > \$200") rather than patterns learnable from transaction features alone.
3. **Random/Manual Discount Application:** If discounts are manually applied by sales staff or given randomly during promotions, no amount of machine learning can predict them from transaction features.
4. **Feature Engineering Limitations:** The encoded categorical variables (Category, Payment Method, Location) may have lost important nuanced information during Label Encoding. One-hot encoding might have preserved more information.
5. **Data Quality Issues:** 33.4% missing values in the target variable initially, along with missing values in other features, suggests potential data quality problems that persist despite imputation.
6. **Temporal Factors Not Captured:** By dropping the Transaction Date, we lost potential seasonal or time-based patterns that might influence discount application.

7.4 Challenges Faced

Technical Challenges:

1. **Class Imbalance:** Required SMOTE implementation to balance classes, but this added synthetic data may not perfectly represent real discount patterns
2. **Weak Feature Signals:** Correlation analysis revealed no strong relationships, making feature selection and engineering extremely difficult
3. **Overfitting vs. Underfitting:** Finding the right model complexity was challenging - simple models underfit while complex models overfit

4. **Missing Data Handling:** 33.4% missing values in the target variable required careful imputation decisions that may have introduced bias
5. **Feature Encoding Decisions:** Choosing between Label Encoding and One-Hot Encoding for categorical variables, with potential information loss in our chosen approach

Interpretability Challenges:

6. **Business Logic Gap:** Difficulty bridging the gap between what the data shows and what business logic might actually govern discount decisions
7. **Model Selection Dilemma:** No clear "best" model due to conflicting metrics - accuracy vs. recall trade-off made selection dependent on business priorities
8. **Result Communication:** Explaining why all models perform poorly without discouraging stakeholders from the approach

Data Challenges:

9. **Limited Feature Set:** Only 6 meaningful features after preprocessing, limiting model learning capacity
10. **Cluster Interpretation:** While K-Means found two clusters, they overlap significantly, making practical business recommendations difficult

7.5 Recommendations for Improvement

Data Collection:

- Gather additional features (customer demographics, purchase history, loyalty status, marketing campaign exposure)
- Include temporal features properly (time of day, day of week, seasonality)
- Capture business context (promotion events, inventory levels, competitor activities)

Modeling Improvements:

- Try ensemble methods (Random Forest, XGBoost, AdaBoost)
- Experiment with One-Hot Encoding instead of Label Encoding
- Implement cross-validation for more robust evaluation
- Use hyperparameter tuning (GridSearchCV) to optimize model parameters
- Consider treating this as a regression problem (predict discount amount) if discount values are available

Business Approach:

- Investigate whether discount decisions follow deterministic business rules that could be explicitly modeled
- Consider using models to identify opportunities for discounts rather than predicting existing patterns
- Focus on the successful customer segmentation for targeted marketing strategies

7.6 Final Remarks

This project successfully demonstrated the complete machine learning pipeline from data preprocessing through model evaluation. While the predictive models achieved limited success due to weak feature-target relationships, the unsupervised learning component successfully identified meaningful customer segments. The analysis reveals that discount application in retail may be driven more by business rules and external factors than by transaction features alone.

The experience highlights an important lesson in machine learning: **not all problems are solvable with the available data**. Sometimes, poor model performance indicates the need for better data rather than better algorithms. The systematic approach, proper preprocessing, and rigorous evaluation methodology established in this project provide a solid foundation for future analyses with enhanced datasets.

Key Takeaway: Success in machine learning requires not just technical proficiency but also domain understanding to recognize when the available data is insufficient for the problem at hand.